

Operator Overloading & the Anatomy of a Copy

Review worksheet — `operator+` / `=`, `const` member functions, copy semantics — builds on Worksheet 02

Name _____ Date _____ Section _____

array[0] Fill in the Blank

one word / token per blank

- Overloading `operator+` as a *member* function works because the left operand of `a + b` is implicitly _____.
- A member function that promises not to modify the object it's called on is marked _____ after its parameter list.
- In `Fraction operator+(const Fraction& rhs) const;` the first `const` protects the parameter `rhs`; the trailing `const` promises not to modify _____.
- Written as a member, `operator+` takes _____ explicit parameter(s) for the right-hand operand; written as a non-member (like `operator<<`), it takes _____.
- Keeping `gcd` and `lcm` in `private:` instead of `public:` is an example of _____ — callers of `Fraction` don't need to know how simplification works.
- `Fraction F2 = F1;` invokes a compiler-generated _____ constructor, which copies each data member one at a time.
- That generated constructor is safe to use here without writing your own, because `Fraction`'s members are plain _____ — there's no owned heap memory to duplicate.
- `operator+=` modifies the calling object and returns _____ so calls like `a += b += c` can chain.

Word bank `*this` `const` `one` `two` `encapsulation` `copy` `ints` `Fraction&`

array[1] Find the Error

continued on back

CASE A

COMPILES? Y / N

```
class Fraction {
    int num, den;
public:
    Fraction(int n, int d)
        : num(n), den(d) {}
};

Fraction operator+(const Fraction& a,
                  const Fraction& b) {
    return Fraction(a.num + b.num,
                  a.den + b.den); // line 9
}
```

Line 9 won't compile. Why not, and what's the one-line fix — added *inside* the class, without touching `operator+` itself?

CASE B

COMPILES? Y / N

```
class Fraction {
    int num, den;
public:
    Fraction(int n, int d)
        : num(n), den(d) {}
    Fraction operator+(const Fraction& rhs) {
        return Fraction(num + rhs.num,
                        den + rhs.den);
    }
};

void show(const Fraction& a,
         const Fraction& b) {
    Fraction c = a + b; // line 13
}
```

Line 13 won't compile even though `operator+` looks right. What's missing from its signature, and why do `a / b` being `const&` trigger the error?

Operator Overloading & the Anatomy of a Copy

Review worksheet — `operator+` / `=`, `const` member functions, copy semantics — builds on Worksheet 02

array[1] Find the Error

2 of 2

CASE C

COMPILES? Y / N

```
bool Fraction::operator==(
    const Fraction& rhs) const {
    return num == rhs.num
        && den == rhs.den;
}

Fraction a(1, 2), b(2, 4);
cout << (a == b);    // prints 0
```

This compiles and runs, but `a` and `b` represent the *same* value. Why does it print `0`, and what's the correct comparison?

CASE D

COMPILES? Y / N

```
class Fraction {
    int num, den;
public:
    Fraction(int n) : num(n), den(1) {}
    Fraction(int n, int d) : num(n), den(d) {}
    Fraction operator+(
        const Fraction& rhs) const {
        return Fraction(num + rhs.num,
            den + rhs.den);
    }
};

Fraction a(1, 2);
Fraction b = a + 2;    // line 12, OK
Fraction c = 2 + a;    // line 13, rejected
```

Line 12 compiles but line 13 doesn't, even though `+` should be commutative. Why does the compiler accept one and reject the other?

CASE E

COMPILES? Y / N

```
class Buffer {
    int* data;
    int size;
public:
    Buffer(int n) : data(new int[n]), size(n) {}
};

int main() {
    Buffer b1(10);
    Buffer b2 = b1;    // line 10 -- compiles fine
                    // crash or corruption here
}
```

`Buffer` has no user-written copy constructor, so line 10 uses the compiler-generated one. What does it do with `data`, and why does the program misbehave when `b1` and `b2` go out of scope? (You don't need to fix it — just diagnose it.)

array[2] Short Answer

full sentences

- Why must `Fraction::operator+` be declared `const` if neither operand should change? Give one line of calling code that would fail to compile if the trailing `const` were missing.

- `operator<<` must be non-member (or `friend`); `operator+` for two `Fraction`s is usually a member. What's different about the left operand in each case that explains this?

- In your own words, why does copying a `Buffer` the default way (Case E) cause trouble? What's the two-word term for a copy that duplicates a pointer's *address* instead of what it points to?

- `Fraction` never needs a hand-written copy constructor, destructor, or `operator=`. What is it about its members that makes the compiler-generated versions safe, and what kind of member (like `Buffer`'s) would force you to write your own?
